QA Engineer Assessment — Response

Candidate: jkp-yusack-dev **Date:** 2025-10-10 **Case:** Moonlight Plaza Associates v. Cancun Farms

---

Task 1 — Identify Issues (15 Found)

Category A: Transcription Errors (ASR Misrecognitions)

#	Issue	Location	Why It Matters
1	**"scientists"** instead of "clients"	Raw ASR: `we represent scientists, Moonlight Plaza Association`	Completely changes the meaning — the firm represents legal clients, not scientists. Critical legal error.
2	**"Moonlite"** instead of "Moonlight"	Raw ASR word: `Moonlite`	Misspells the plaintiff's name. Undermines credibility of the transcript.
3	**"councel"** instead of "counsel"	Raw ASR word: `councel.`	Misspells a key legal term.
4	**"Richman & Lavine"** instead of "Richmond & Levine"	Processed transcript: `Richman & Lavine PC` vs. raw ASR: `Richmond and Levine`	The AI processing introduced a new error. The raw ASR was correct ("Richmond and Levine") but the AI "corrected" it to the wrong name.
5	**"Cancun Farms"** / **"Cancun Forms"** instead of "Cannon Farms"	Processed transcript: multiple instances of `Cancun Farms`, `Cancun Forms 2`	Consistently misnames the defendant party. "Cancun" is a city in Mexico — not a farm company.
6	**"No Further Action leather"** instead of "No Further Action letter"	Processed transcript	A "No Further Action letter" is a specific regulatory document. "Leather" changes the meaning entirely.
7	**"Professional Geologists"** instead of "Professional Geologist"	Processed transcript (suffix PG)	PG = Professional Geologist (singular). The plural form is incorrect for a single-person suffix.
8	**"Miss Jacob"** instead of "Ms. Jacob"	Raw ASR word: `Miss`	ASR misrecognized "Ms." as "Miss." Minor but shows ASR quality issues with honorifics.

Category B: Speaker Attribution Issues

#	Issue	Location	Why It Matters
9	**Speaker labels swapped in metadata**	`metadata_corrupted.json`: Speaker B (Chris Jacob, PLAINTIFF ATTORNEY) has `real_time_asr_label: "THE WITNESS"`. Speaker A (Terry/Jerry	Speaker B is labeled as PLAINTIFF ATTORNEY but Chris Jacob is the witness (project manager at Cannon Farms). Speaker A is labeled WITNESS

		Sellingman, WITNESS) has `real_time_asr_label: "MS SELIGMAN"`	but Jerry Sellingman is the attorney. **Roles are completely reversed.**
10	**"MS. ATRIANO"** appears in processed transcript	Multiple lines in processed transcript	No speaker named "Ms. Atriano" exists in the metadata. This is an unknown/unmapped speaker label — could be the defendant's attorney objecting, but the system has no record of this person.
11	**Witness answers attributed to wrong speaker**	Processed transcript: `WITNESS: I am a senior environmental project manager. And what are your duties...`	The question "And what are your duties..." is the attorney's question, not the witness's answer. It's incorrectly lumped under the WITNESS label.
12	**"WITNESS" label used inconsistently**	Processed transcript uses both `WITNESS` and `MS SELIGMAN` for the same person at different points	Inconsistent label normalization — the AI sometimes uses the role-based label and sometimes the name-based label.
13	**Name mismatch in metadata Speaker A**	`first_name: "Terry"`, `last_name: "Sellingman"`, but `full_name: "Jerry Sellingman"`	First name and full name disagree. The ASR correctly identifies "Jerry Sellingman" but the first_name field says "Terry."

Category C: Timestamp Inconsistencies

#	Issue	Location	Why It Matters
14	**Word timestamps precede utterance start time**	Raw ASR: Utterance start=`17920`, but first word "Good" starts at `16000`	A word cannot start before its containing utterance. This indicates a data integrity bug in the ASR pipeline.
15	**Duplicate word timestamps**	Raw ASR: Words "I" and "am" both have `start: 4560`; words "a" and "little" both have `start: 38240`; words "you" and "do" both have `start: 174080`	Two distinct words cannot occupy the exact same millisecond. This breaks any time-based indexing or playback sync.
16	**`created_at` timestamp is invalid**	Metadata: `"created_at": "2025-10-09T26:61:00Z"`	Hour `26` and minute `61` are both invalid. This would crash any parser expecting valid ISO 8601.
17	**Audio/transcript duration mismatch**	Metadata: `audio_duration_ms: 630000` (10:30) vs. `transcript_duration_ms: 605000` (10:05)	25-second gap is unaccounted for. Could mean missing content or a processing error.

Category D: Formatting / Structural Problems

#	Issue	Location	Why It Matters
18	**Duplicate lines in processed transcript**	`THE REPORTER: Absolutely.` appears twice; `THE REPORTER: Good morning...` appears in both timestamped and non-timestamped sections	Duplication inflates the transcript and confuses readers. Indicates a bug in the AI deduplication logic.
19	**`"error"` field is a string, not null**	Metadata: `"error": "null"`	Should be `null` (JSON null) or omitted. A string `"null"` would pass a truthy check in many languages, masking the absence of an error.
20	**Transcript ends mid-sentence**	Processed transcript ends with `MS SELIGMAN: And going.`	An incomplete final utterance indicates truncated processing or a stream cutoff bug.

Task 2 — Test Cases (10 Designed)

TC-1: ASR Word Accuracy — Known Entity Names

Field	Value
Description	Verify that known entity names (party names, firm names) are transcribed correctly
Input	Raw ASR transcript containing "Moonlight Plaza Association", "Cannon Farms", "Richmond & Levine"
Expected Output	All entity names match the metadata parties list exactly
Failure Condition	Any entity name has Levenshtein distance > 0 from the canonical name in metadata

TC-2: Speaker Role Consistency

Field	Value
Description	Verify that speaker roles in metadata are internally consistent (first_name + last_name = full_name)
Input	`metadata_corrupted.json` speakers array
Expected Output	`full_name === first_name + " " + last_name` for all speakers
Failure Condition	Any speaker has mismatched name components

TC-3: Timestamp Monotonicity Within Utterances

Field	Value
Description	Verify that word timestamps within an utterance are monotonically non-decreasing
Input	Raw ASR utterance with word-level timestamps
Expected Output	For each utterance, `words[i].start <= words[i+1].start` for all i
Failure Condition	Any word has a timestamp earlier than the previous word

TC-4: Utterance-Word Timestamp Containment

Field	Value
Description	Verify that all word timestamps fall within their parent utterance's time range
Input	Raw ASR utterance with `start` time and word array
Expected Output	`word.start >= utterance.start` for all words
Failure Condition	Any word starts before its utterance

TC-5: Duplicate Timestamp Detection

Field	Value
Description	Detect words with identical timestamps within the same utterance
Input	Raw ASR utterance word array
Expected Output	No two consecutive words share the exact same timestamp
Failure Condition	Any pair of words has `word[i].start === word[i+1].start`

TC-6: ISO 8601 Date Validation

Field	Value
Description	Verify all date/time fields in metadata are valid ISO 8601
Input	`metadata_corrupted.json` — `created_at` field
Expected Output	`Date.parse(created_at)` returns a valid date; hours 0-23, minutes 0-59
Failure Condition	`isNaN(Date.parse(created_at))` or extracted hour > 23 or minute > 59

TC-7: Audio-Transcript Duration Reconciliation

Field	Value
Description	Verify transcript duration does not exceed audio duration

Input	Metadata `audio_duration_ms` and `transcript_duration_ms`
Expected Output	`transcript_duration_ms <= audio_duration_ms`
Failure Condition	`transcript_duration_ms > audio_duration_ms` or gap > threshold (e.g., 5000ms)

TC-8: Speaker Label Mapping Completeness

Field	Value
Description	Verify every speaker label in the transcript has a corresponding entry in metadata
Input	Set of unique speaker labels from transcript + metadata speakers array
Expected Output	Every transcript speaker label matches a `post_asr_label` or `real_time_asr_label` in metadata
Failure Condition	Any transcript speaker label (e.g., "MS. ATRIANO") has no metadata mapping

TC-9: Duplicate Line Detection in Processed Transcript

Field	Value
Description	Detect consecutive duplicate lines in the processed transcript
Input	Processed transcript text file
Expected Output	No two consecutive non-empty lines are identical
Failure Condition	Any line N === line N+1 (after trimming whitespace)

TC-10: Transcript Completeness Check

Field	Value
Description	Verify the transcript does not end with a sentence fragment
Input	Last line of processed transcript
Expected Output	Last utterance ends with terminal punctuation (. ! ?) or is a complete sentence
Failure Condition	Last line ends with a conjunction or incomplete phrase (e.g., "And going.")

Task 3 — API Testing Plan

Endpoint: `POST /transcripts/process`

Purpose

Submits raw ASR output + metadata for processing; returns a processed transcript.

Test Strategy

Happy Path:

- Submit valid raw ASR JSON + valid metadata → expect `200 OK` with processed transcript
- Verify response contains all utterances from input
- Verify speaker labels are normalized consistently

Validation Rules:

Rule	Test
Required fields	Omit `utterances` → expect `400 Bad Request`
Required fields	Omit `speakers` in metadata → expect `400`
Timestamp format	Submit invalid timestamps (negative, non-numeric) → expect `400`
Speaker mapping	Submit transcript with unmapped speaker labels → expect `400` or warning
Empty payload	Submit `{}` → expect `400`
Content-Type	Submit without `application/json` → expect `415`

Edge Cases:

- Single-word utterance
- Very long transcript (1000+ utterances) — test performance/timeouts
- Overlapping speaker timestamps (two speakers at same time)
- Zero-duration audio (empty file)
- Unicode/special characters in transcript text
- Null/missing word-level timestamps
- Duplicate speaker labels in metadata

Response Validation:

- Response schema matches expected format
- All input utterances are represented in output
- No hallucinated content (output doesn't add words not in input)
- Speaker labels are consistently normalized

Endpoint: `GET /transcripts/:id`

Purpose

Retrieves a previously processed transcript by ID.

Test Strategy

Happy Path:

- Submit valid transcript ID → expect `200 OK` with transcript data
- Verify returned data matches what was stored

Validation Rules:

Rule	Test
Valid ID format	Submit malformed ID (e.g., "abc-123!") → expect `400`
Missing ID	Submit empty path → expect `400` or `404`

Non-existent ID	Submit valid-format but non-existent ID → expect `404`

Edge Cases:

- ID with SQL injection attempts (`1; DROP TABLE`)
- ID with XSS attempts (`<script>alert(1)</script>`)
- Extremely long ID string
- Concurrent requests for same ID — verify consistent response
- Transcript still processing (if async) → expect `202 Accepted` or appropriate status
- Deleted/archived transcript → expect `404` or `410 Gone`

Response Validation:

- Include `Content-Type: application/json`
- Include transcript metadata (created_at, duration, speaker count)
- Include word-level timestamps if requested
- Consistent response across repeated calls (idempotency)

Task 4 — Workflow Testing

State Machine: `NEW → ASSIGNED → TRANSCRIBED → REVIEWED → COMPLETED`

Valid Transition Tests

Test	From	Action	To	Expected
WT-1	NEW	Assign to user	ASSIGNED	`200`, status = ASSIGNED
WT-2	ASSIGNED	Submit transcript	TRANSCRIBED	`200`, status = TRANSCRIBED
WT-3	TRANSCRIBED	Submit review	REVIEWED	`200`, status = REVIEWED
WT-4	REVIEWED	Mark complete	COMPLETED	`200`, status = COMPLETED

Invalid Transition Tests

Test	From	Action	To	Expected
WT-5	NEW	Submit transcript	—	`409 Conflict` — must be ASSIGNED first
WT-6	NEW	Mark complete	—	`409 Conflict` — cannot skip steps
WT-7	ASSIGNED	Mark complete	—	`409 Conflict` — must be TRANSCRIBED first
WT-8	TRANSCRIBED	Mark complete	—	`409 Conflict` — must be REVIEWED first
WT-9	COMPLETED	Re-assign	—	`409 Conflict` — terminal state
WT-10	COMPLETED	Re-transcribe	—	`409 Conflict` — terminal state

Edge Case Tests

Test	Scenario	Expected

WT-11	Reassignment in ASSIGNED state	Old assignee notified, new assignee set, status stays ASSIGNED
WT-12	Skip from ASSIGNED → REVIEWED	`409 Conflict` — TRANSCRIBED step cannot be skipped
WT-13	Concurrent state transitions	Only one succeeds; other gets `409 Conflict`
WT-14	Reopen COMPLETED transcript	Either `409` (immutable) or creates new workflow instance
WT-15	Timeout in TRANSCRIBED state	Auto-escalation or notification after threshold
WT-16	Review rejection (REVIEWED → TRANSCRIBED)	Status reverts to TRANSCRIBED with rejection notes
WT-17	Audit trail	Every transition logs: timestamp, actor, previous state, new state

---

Task 5 — Automation Approach

Tools

Category	Tool
Test Framework	**pytest** (Python) or **Jest** (Node.js)
API Testing	**pytest + requests** or **Supertest**
Assertions	**pytest** built-in or **Chai**
CI/CD	**GitHub Actions** or **Jenkins**
Linting	**flake8** / **ESLint**
Coverage	**pytest-cov** / **nyc**
Mocking	**pytest-mock** / **MSW** (Mock Service Worker)

Example Test Structure (pytest)

```
# tests/test_transcript_processing.py

import pytest
import json
import requests
from datetime import datetime

BASE_URL = "http://localhost:8080"
```

```python
class TestTranscriptValidation:
    """Task 1 & 2: Transcript quality checks"""

    @pytest.fixture
    def raw_transcript(self):
        with open("transcript_raw_corrupted.json") as f:
            return json.load(f)

    @pytest.fixture
    def metadata(self):
        with open("metadata_corrupted.json") as f:
            return json.load(f)

    def test_speaker_name_consistency(self, metadata):
        """TC-2: first_name + last_name must equal full_name"""
        for speaker in metadata["speakers"]:
            expected = f"{speaker['first_name']} {speaker['last_name']}"
            assert speaker["full_name"] == expected, (
                f"Name mismatch: '{speaker['full_name']}' != '{expected}'"
            )

    def test_word_timestamps_monotonic(self, raw_transcript):
        """TC-3: Word timestamps must be non-decreasing within utterance"""
        for utterance in raw_transcript["utterances"]:
            prev_ts = -1
            for word in utterance["words"]:
                assert word["start"] >= prev_ts, (
                    f"Non-monotonic timestamp: {word['start']} < {prev_ts}"
                )
                prev_ts = word["start"]

    def test_word_within_utterance_bounds(self, raw_transcript):
        """TC-4: Words must not start before their utterance"""
        for utterance in raw_transcript["utterances"]:
            for word in utterance["words"]:
                assert word["start"] >= utterance["start"], (
                    f"Word '{word['text']}' starts at {word['start']} "
                    f"before utterance at {utterance['start']}"
                )

    def test_no_duplicate_word_timestamps(self, raw_transcript):
        """TC-5: No two words share the same timestamp"""
        for utterance in raw_transcript["utterances"]:
            seen = set()
            for word in utterance["words"]:
                assert word["start"] not in seen, (
                    f"Duplicate timestamp {word['start']} for '{word['text']}'"
                )
                seen.add(word["start"])

    def test_created_at_is_valid_iso8601(self, metadata):
        """TC-6: created_at must be valid ISO 8601"""
```

```
        dt = datetime.fromisoformat(metadata["created_at"].replace("Z", "+00:00"))
        assert 0 <= dt.hour <= 23
        assert 0 <= dt.minute <= 59

    def test_audio_transcript_duration_consistency(self, metadata):
        """TC-7: Transcript must not exceed audio duration"""
        assert metadata["transcript_duration_ms"] <= metadata["audio_duration_ms"]


class TestAPIEndpoints:
    """Task 3: API testing"""

    def test_post_process_valid_payload(self):
        payload = {"utterances": [], "metadata": {}}
        resp = requests.post(f"{BASE_URL}/transcripts/process", json=payload)
        assert resp.status_code in [200, 202]

    def test_post_process_missing_utterances(self):
        payload = {}
        resp = requests.post(f"{BASE_URL}/transcripts/process", json=payload)
        assert resp.status_code == 400

    def test_get_transcript_valid_id(self):
        resp = requests.get(f"{BASE_URL}/transcripts/abc-123")
        assert resp.status_code in [200, 404]

    def test_get_transcript_invalid_id(self):
        resp = requests.get(f"{BASE_URL}/transcripts/; DROP TABLE transcripts;")
        assert resp.status_code in [400, 404]


class TestWorkflowTransitions:
    """Task 4: State machine testing"""

    VALID_TRANSITIONS = [
        ("NEW", "ASSIGNED"),
        ("ASSIGNED", "TRANSCRIBED"),
        ("TRANSCRIBED", "REVIEWED"),
        ("REVIEWED", "COMPLETED"),
    ]

    def test_valid_transitions(self):
        for from_state, to_state in self.VALID_TRANSITIONS:
            resp = requests.post(
                f"{BASE_URL}/workflow/transition",
                json={"state": from_state, "next_state": to_state},
            )
            assert resp.status_code == 200, (
                f"Valid transition {from_state} → {to_state} failed"
            )

    def test_invalid_skip_transitions(self):
```

```
        invalid = [
            ("NEW", "COMPLETED"),
            ("NEW", "TRANSCRIBED"),
            ("ASSIGNED", "REVIEWED"),
            ("COMPLETED", "ASSIGNED"),
        ]
        for from_state, to_state in invalid:
            resp = requests.post(
                f"{BASE_URL}/workflow/transition",
                json={"state": from_state, "next_state": to_state},
            )
            assert resp.status_code == 409, (
                f"Invalid transition {from_state} → {to_state} should be rejected"
            )
```

CI/CD Integration (GitHub Actions)

```yaml
# .github/workflows/test.yml
name: QA Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: pip install pytest requests pytest-cov
      - name: Run transcript quality tests
        run: pytest tests/test_transcript_processing.py -v --cov
      - name: Run API tests (against mock server)
        run: pytest tests/test_api.py -v
      - name: Run workflow tests
        run: pytest tests/test_workflow.py -v
```

Bonus — Scoring System & Programmatic Detection

Transcript Quality Score (0–100)

Category	Weight	Checks
Word Accuracy	30 pts	Levenshtein distance on known entities; flag words with confidence < 0.7
Speaker Attribution	25 pts	All labels mapped to metadata; no unknown speakers; consistent labeling

Timestamp Integrity	20 pts	Monotonic timestamps; no duplicates; words within utterance bounds
Completeness	15 pts	No truncated final utterance; audio/transcript duration match
Formatting	10 pts	No duplicate lines; valid JSON; valid ISO dates

Score = sum of category scores, capped at 100.

Programmatic Detection Rules (Pseudocode)

```
def detect_issues(raw_asr, processed, metadata):
    issues = []

    # Rule 1: Name consistency
    for speaker in metadata["speakers"]:
        if f"{speaker['first_name']} {speaker['last_name']}" !=
speaker["full_name"]:
            issues.append("SPEAKER_NAME_MISMATCH")

    # Rule 2: Timestamp monotonicity
    for utterance in raw_asr["utterances"]:
        for i in range(1, len(utterance["words"])):
            if utterance["words"][i]["start"] < utterance["words"][i-1]["start"]:
                issues.append("NON_MONOTONIC_TIMESTAMP")
            if utterance["words"][i]["start"] == utterance["words"][i-1]["start"]:
                issues.append("DUPLICATE_TIMESTAMP")

    # Rule 3: Speaker label coverage
    transcript_labels = set(u["speaker"] for u in raw_asr["utterances"])
    metadata_labels = set(
        s["post_asr_label"] for s in metadata["speakers"]
    ) | set(
        s["real_time_asr_label"] for s in metadata["speakers"]
    )
    unmapped = transcript_labels - metadata_labels
    for label in unmapped:
        issues.append(f"UNMAPPED_SPEAKER:{label}")

    # Rule 4: Entity name matching
    known_entities = [p for p in metadata.get("parties", [])]
    processed_text = "\n".join(processed.split("\n"))
    for entity in known_entities:
        if entity.lower() not in processed_text.lower():
            issues.append(f"MISSING_ENTITY:{entity}")

    # Rule 5: Date validation
    try:
        dt = datetime.fromisoformat(metadata["created_at"].replace("Z", "+00:00"))
        if dt.hour > 23 or dt.minute > 59:
            issues.append("INVALID_TIMESTAMP")
```

```
    except ValueError:
        issues.append("INVALID_TIMESTAMP")

    # Rule 6: Duplicate consecutive lines
    lines = processed.strip().split("\n")
    for i in range(1, len(lines)):
        if lines[i].strip() == lines[i-1].strip() and lines[i].strip():
            issues.append("DUPLICATE_LINE")

    # Rule 7: Truncated ending
    last_line = lines[-1].strip()
    if last_line and not last_line[-1] in ".!?":
        issues.append("TRUNCATED_TRANSCRIPT")

    return issues
```

Summary

Task	Deliverable	Count
Task 1	Issues identified	**20 issues** across 4 categories
Task 2	Test cases designed	**10 test cases**
Task 3	API testing plan	Full plan for both endpoints
Task 4	Workflow tests	**17 transition tests** (valid, invalid, edge cases)
Task 5	Automation approach	pytest framework + CI/CD pipeline
Bonus	Scoring + detection	100-point rubric + rule-based detector